



Universidad  
Zaragoza

# Other approaches to DRL (model based, offline...)

Applications of Deep Learning  
(69158)



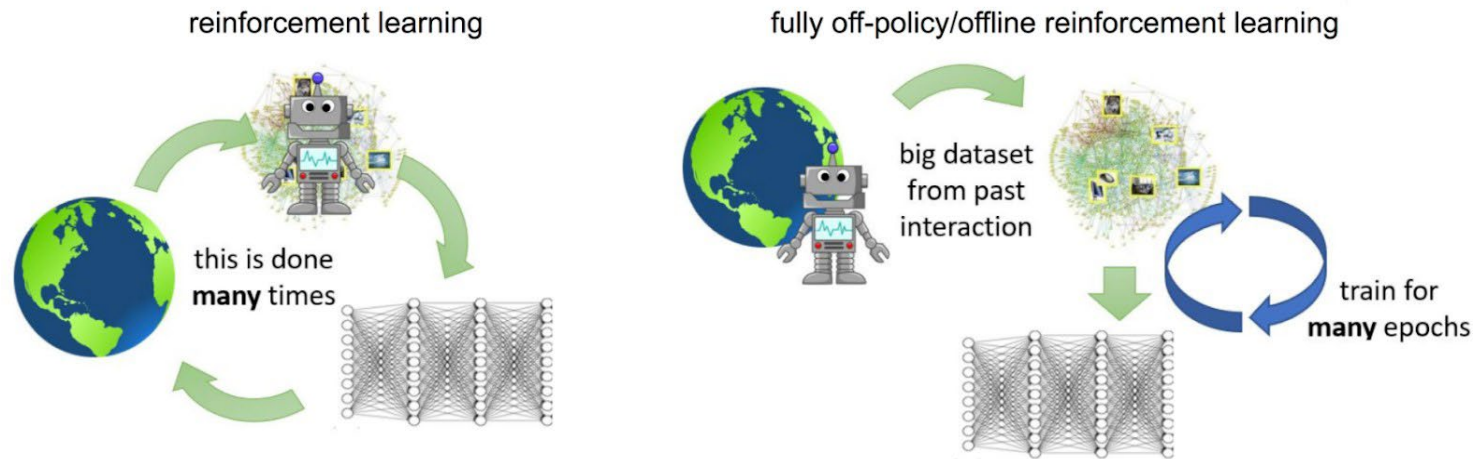
Universidad  
Zaragoza

**Rubén Martínez Cantín**

Dpto. Informática e Ingeniería de Sistemas.

# Offline(Batch) Reinforcement Learning

- Learn from a previously collected static dataset
  - Large static datasets of meaningful behaviors already exist
  - Large datasets at the core of successes in Vision and NLP

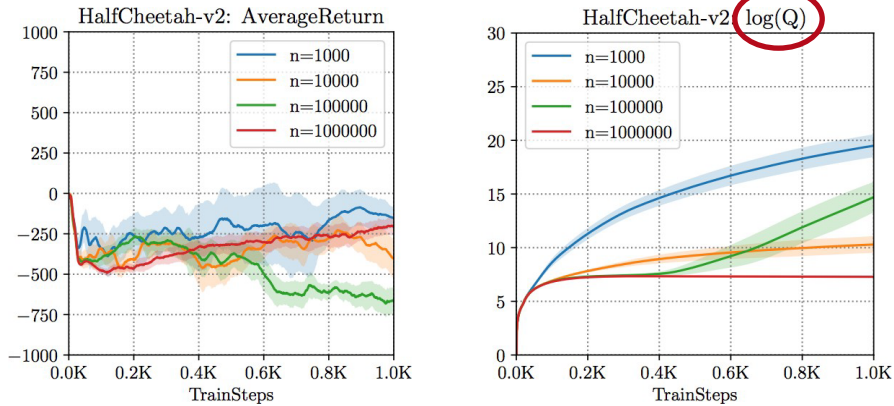


- Active data gathering can be unsafe or expensive in real world problems.
  - Would you look for a tasty mushroom by trial and error or would you check a book?
  - Would you implement epsilon-greedy in a self-driving car?

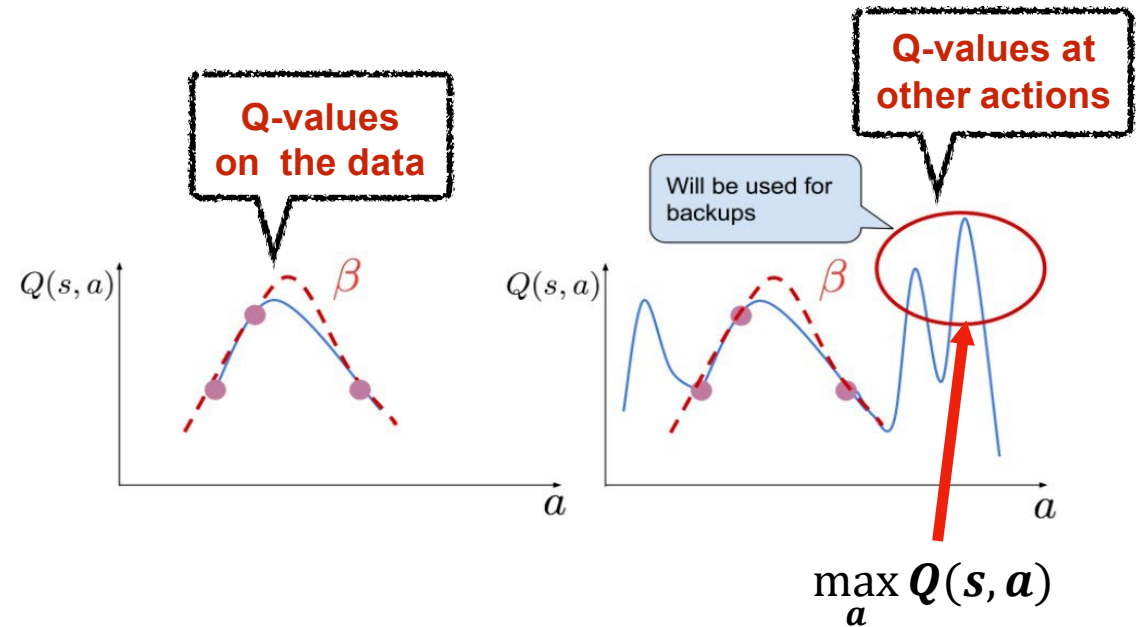


# Offline RL: Problems and solutions

## Q function diverges but policy is not improved

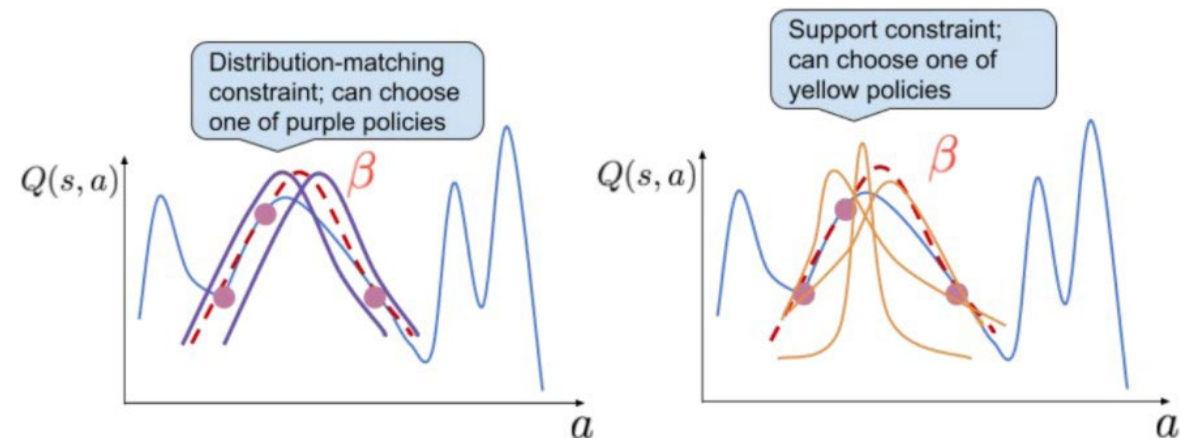


Kumar 2020.



## Solutions:

- Policy Constraint Methods:
- Model-based algorithms
- Direct Q-function penalties (e.g.: Conservative Q-Learning)



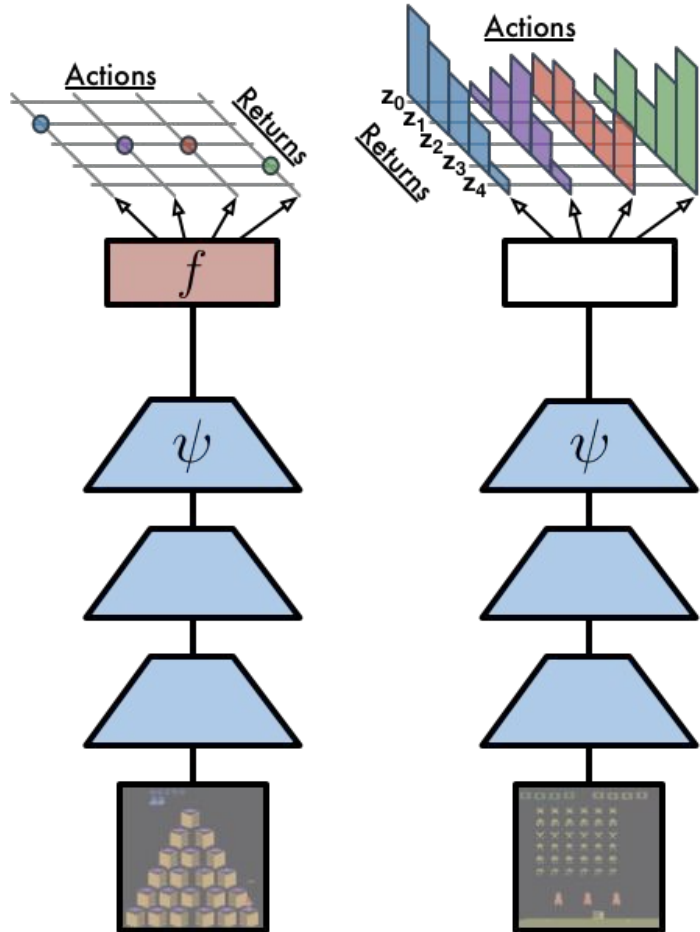
# Distributional Reinforcement Learning



<https://www.distributional-rl.org/>

$$Q(\mathbf{o}, \mathbf{a}; \boldsymbol{\theta})$$

$$f_i(\mathbf{o}, \mathbf{a}; \boldsymbol{\theta})$$



$$\mathbf{Z}(\mathbf{o}, \mathbf{a}; \boldsymbol{\theta}) = z_i \quad w.p. \quad p_i(\mathbf{o}, \mathbf{a}) = \frac{e^{f_i(\mathbf{o}, \mathbf{a}; \boldsymbol{\theta})}}{\sum_j e^{f_j(\mathbf{o}, \mathbf{a}; \boldsymbol{\theta})}}$$

$$\hat{\mathbf{a}} = \arg \max_{\mathbf{a}} Q(\mathbf{o}, \mathbf{a}; \boldsymbol{\theta})$$

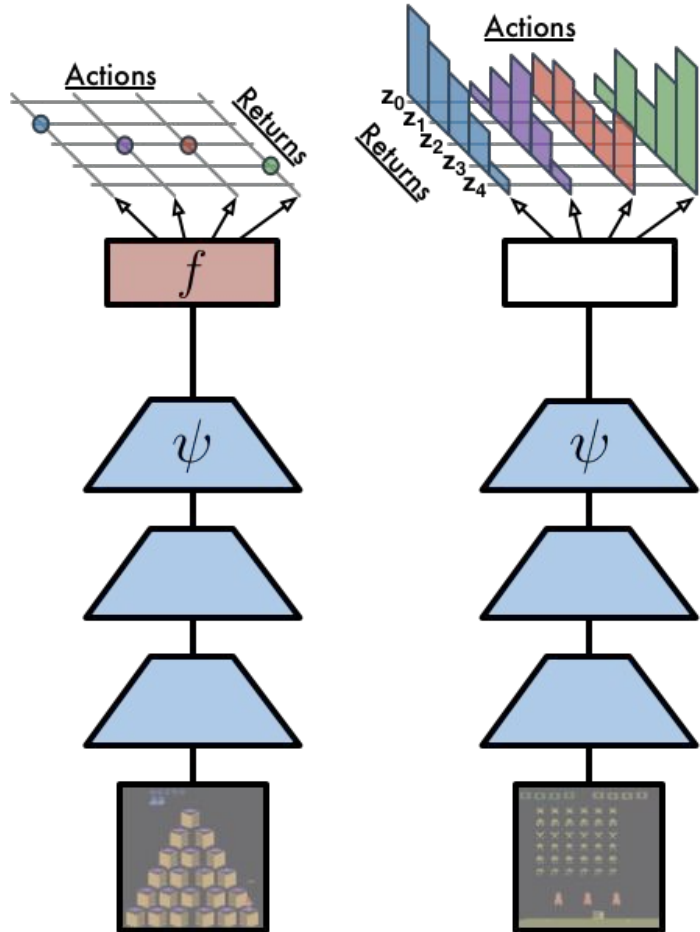
$$= \arg \max_{\mathbf{a}} \mathbb{E} [\mathbf{Z}(\mathbf{o}, \mathbf{a}; \boldsymbol{\theta})]$$

$$\approx \arg \max_{\mathbf{a}} \sum_{i=0}^{N-1} z_i p_i(\mathbf{o}, \mathbf{a})$$

# Distributional Reinforcement Learning

$$Q(o, a; \theta)$$

$$f_i(o, a; \theta)$$



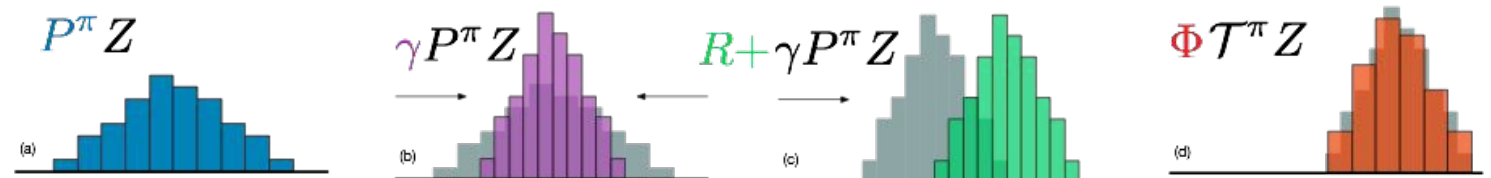
Q-learning loss:

$$\left( r(o, a, o') + \gamma \max_{a'} Q(o', a', \theta^*) - Q(o, a, \theta_i) \right)^2$$

Distributional RL loss:

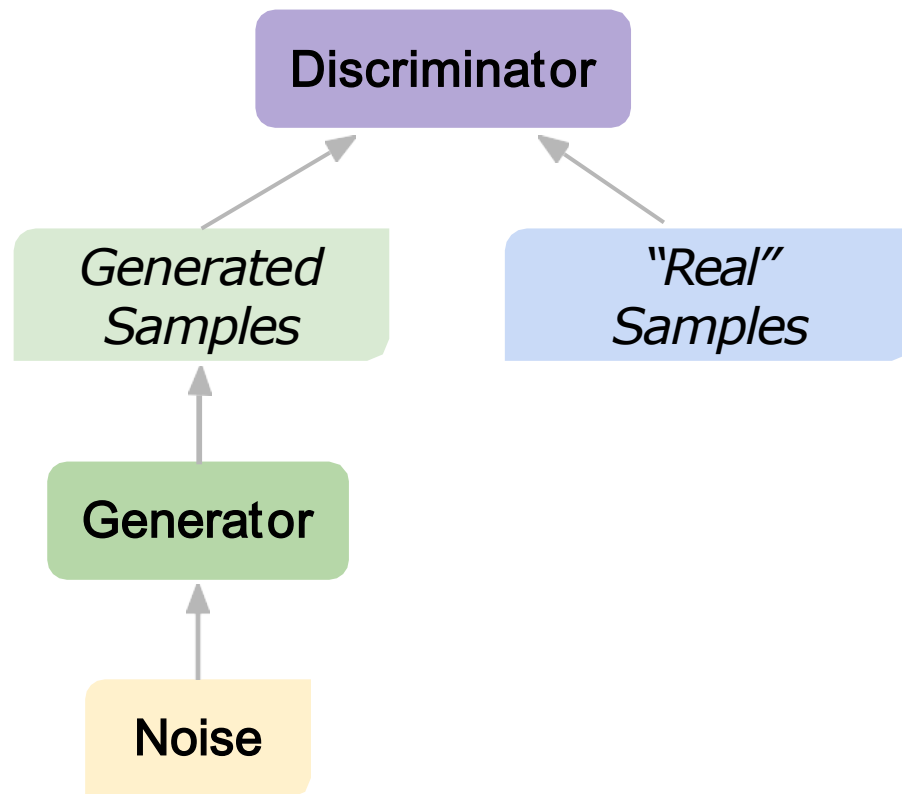
$$Dist \left( r(o, a, o') + \gamma \max_{a'} Z(o', a', \theta^*); Z(o, a, \theta_i) \right)$$

What can be here?

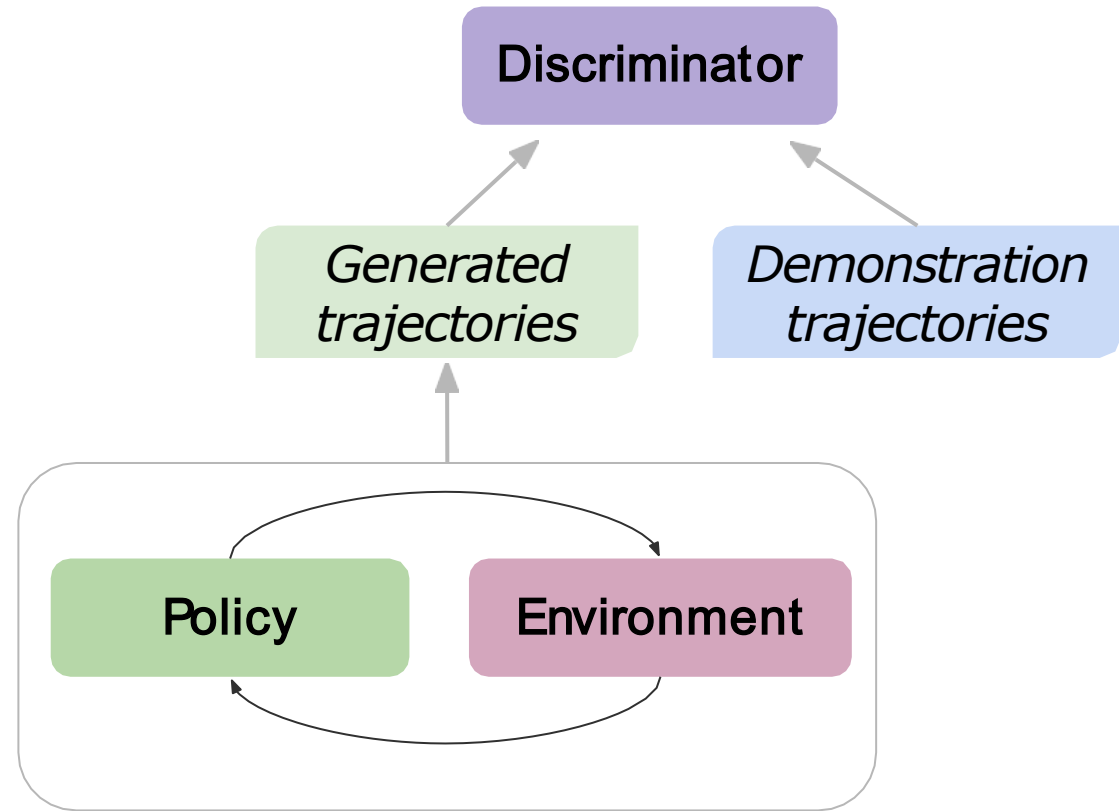


# Generative Adversarial Imitation Learning (GAIL)

## GAN

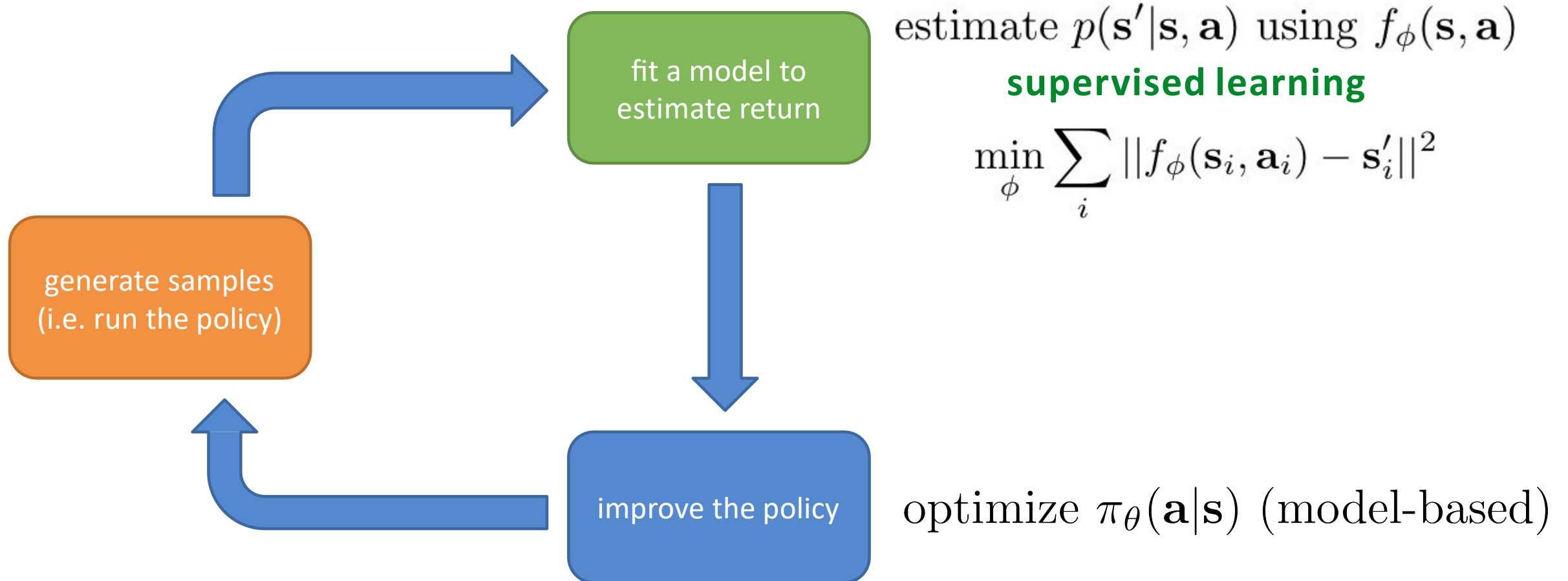


## GAIL



# Model-based Reinforcement Learning

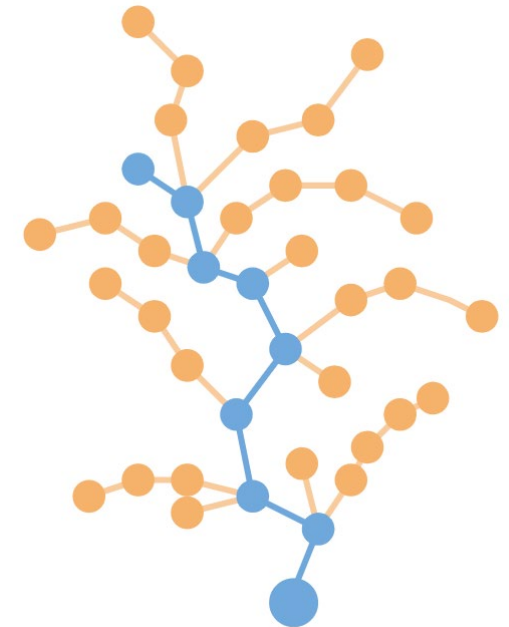
Main idea: learn model of environment





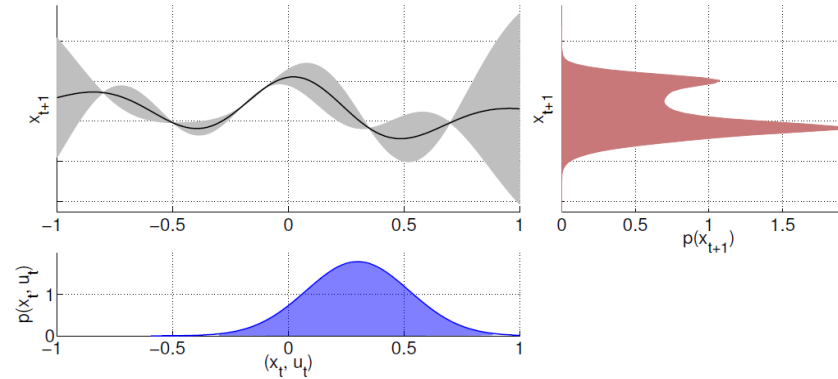
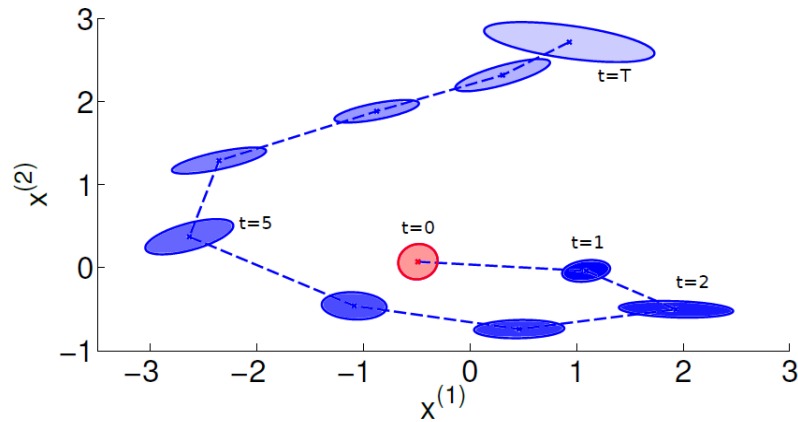
# Model-based Reinforcement Learning

- Often leads to better efficiency
  - If we have the model, we can do planning (dynamic programming, policy/value iteration...)
  - We can use it as a cheap simulator
  - Mix **real** and **model-generated** experience & apply traditional model-free methods
- Model can be reused
  - The model is for the system, not the task.
  - We can use the same model for walking or jumping.
- Examples:
  - Planning with real robots (too expensive, too risky)
  - Simulating complex physical dynamics (too expensive)
  - Interactions with humans (no access)





# PILCO (GP-based MBRL)



Assuming quadratic cost:  
 $c(\mathbf{x}) = 1 - \exp(-\|\mathbf{x} - \mathbf{x}_{\text{target}}\|^2)$

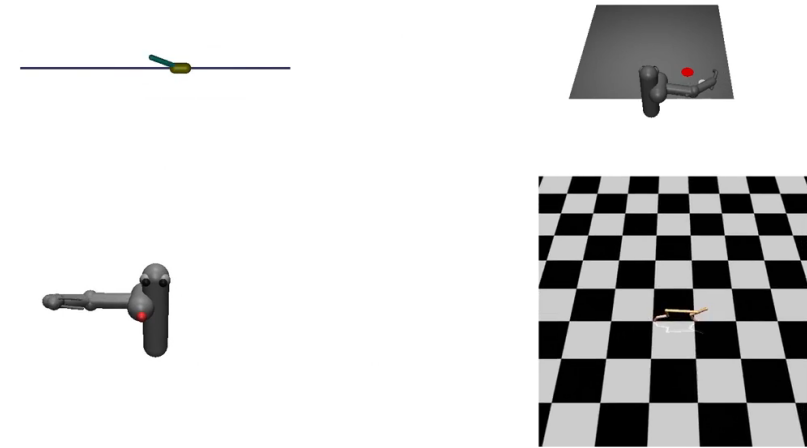
**Policy iteration/search:**  
 Compute analytically  $\nabla_{\theta} J(\theta)$

$$p(\mathbf{x}_{t+1}|\theta) = \iiint \underbrace{p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)}_{\text{GP prediction}} \underbrace{p(\mathbf{x}_t, \mathbf{u}_t|\theta)}_{\mathcal{N}(\mu, \Sigma)} d\mathbf{f} d\mathbf{x}_t d\mathbf{u}_t$$

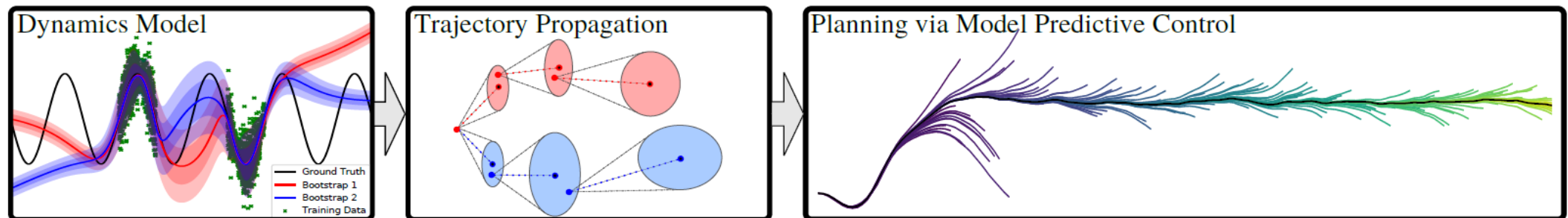
trial #1 (random actions)

# Bayesian DRL with MPC

- Probabilistic Model:
  - GPs can be replaced with Bayesian NN.
- Model Predictive Control:
  - Build a model and plan for T steps.
  - Take first action of plan and gather data.
  - Rebuild model and plan for the next T steps.
- Cross-Entropy Method (CEM) → See value-based methods (slide 34)



Performance reached with  
PE-TS within 100 Trials



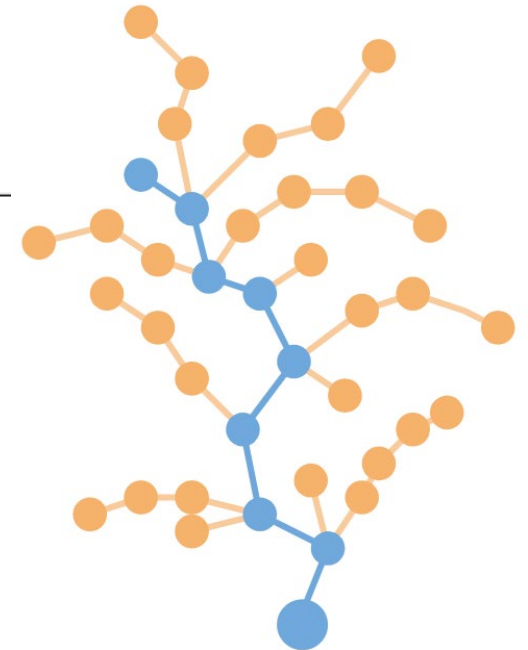
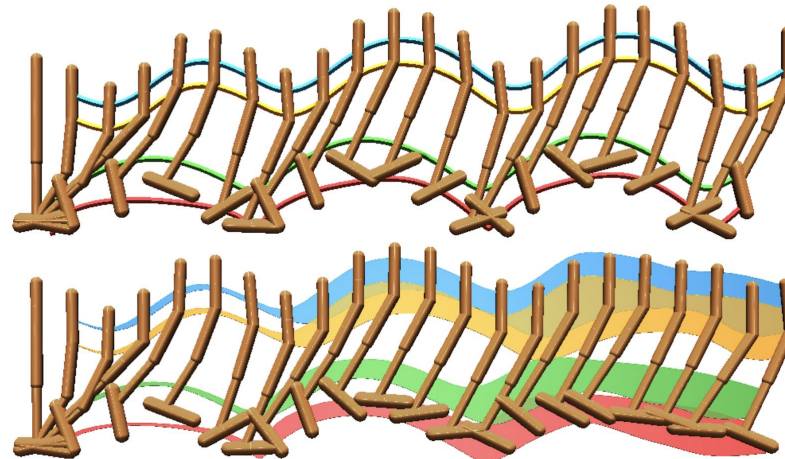
# Bayesian DRL with Actor-Critic

---

**Algorithm 2** Model-Based Policy Optimization with Deep Reinforcement Learning

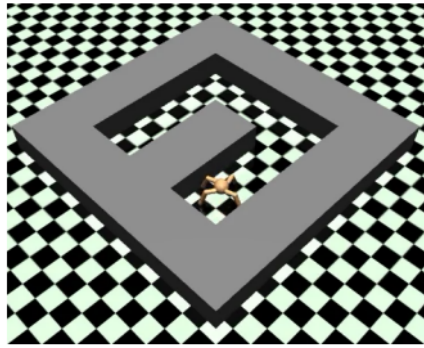
---

- 1: Initialize policy  $\pi_\phi$ , predictive model  $p_\theta$ , environment dataset  $\mathcal{D}_{\text{env}}$ , model dataset  $\mathcal{D}_{\text{model}}$
  - 2: **for**  $N$  epochs **do**
  - 3:   Train model  $p_\theta$  on  $\mathcal{D}_{\text{env}}$  via maximum likelihood
  - 4:   **for**  $E$  steps **do**
  - 5:     Take action in environment according to  $\pi_\phi$ ; add to  $\mathcal{D}_{\text{env}}$
  - 6:     **for**  $M$  model rollouts **do**
  - 7:       Sample  $s_t$  uniformly from  $\mathcal{D}_{\text{env}}$
  - 8:       Perform  $k$ -step model rollout starting from  $s_t$  using policy  $\pi_\phi$ ; add to  $\mathcal{D}_{\text{model}}$
  - 9:     **for**  $G$  gradient updates **do**
  - 10:       Update policy parameters on model data:  $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi, \mathcal{D}_{\text{model}})$
- 

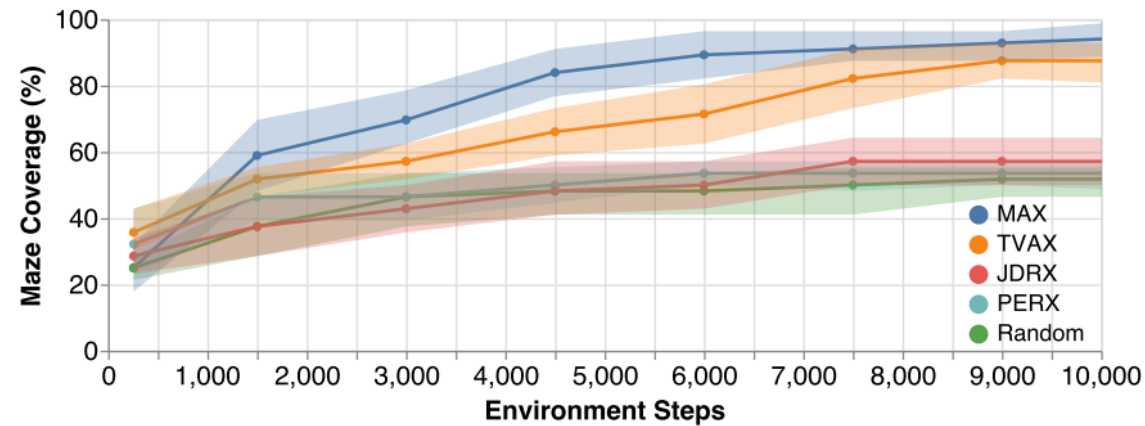


# Bayesian DRL with Actor-Critic for exploration

- Replace task reward with information-gain about model.



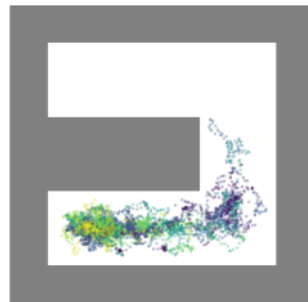
(a) Ant Maze Environment



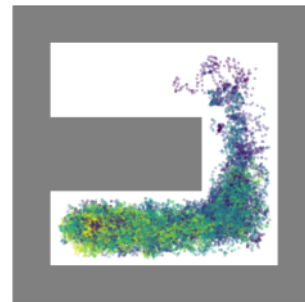
(b) Maze Exploration Performance



(c) 300 steps



(d) 600 steps



(e) 3600 steps



(f) 12000 steps



- 23 RL-enabled robots over two years in Google office buildings to sort waste and recycling



# Agile locomotion



# References

- Main reference:

- Sergey Levine, Deep Reinforcement Learning, CS 285 at UC Berkeley
  - <http://rail.eecs.berkeley.edu/deeprlcourse/>

- Other courses:

- David Silver, Reinforcement Learning, UCL
  - <https://www.davidsilver.uk/teaching/>
- Nando de Freitas, Machine learning, Oxford
  - <https://www.cs.ox.ac.uk/people/nando.defreitas/machinelearning/>

- Books:

- Sutton & Barto, Reinforcement Learning: An Introduction
  - <http://incompleteideas.net/book/the-book-2nd.html>
- Csaba Szepesvári, Algorithms of Reinforcement Learning
  - <https://sites.ualberta.ca/~szepesva/rlbook.html>





Universidad  
Zaragoza



# Applications of Deep Learning (69158)

Máster in Robotics, Graphics  
and Computer Vision

